HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG — CƠ SỞ TP.HCM

Khoa Công nghệ Thông tin 2

Bài thực hành — Buổi 1

Chương 1 & 2 · Cấu trúc dữ liệu và Giải thuật (INT1306_CLC)

Tài liệu dành cho sinh viên · Học kỳ 1, năm học 2026–2027

Lab trên lớp 1A, 2A · Lab về nhà 1B, 2B

Mã nguồn: **C++17** và **Python 3.8+** — mỗi bài đều có file khung (starter) chạy được ngay

Giảng viên: Lê Ngọc Hiếu — lnhieu@ptit.edu.vn

Mục lục

Phần 0. Hướng dẫn chung

0.1 Tổng quan các bài thực hành
0.2 Chuẩn bị môi trường
0.3 Cách làm một bài lab
0.4 Nộp bài và đánh giá

Phần 1. Lab trên lớp — Buổi 1 (Chương 1 và Chương 2)

Lab 1A — Đặc tả thuật toán và phân tích độ phức tạp
Lab 2A — Sắp xếp và tìm kiếm

Phần 2. Lab về nhà — Chương 1 và 2

Lab 1B — Độ phức tạp thuật toán
Lab 2B — Sắp xếp và tìm kiếm (nâng cao)

Phụ lục A. Mẫu phiếu báo cáo

Phụ lục B. Lỗi thường gặp

Phần 0. Hướng dẫn chung

0.1 Tổng quan các bài thực hành

Theo đề cương học phần, phần thực hành bám theo bốn chương lý thuyết. Mỗi chương có **một bài lab trên lớp** (làm có giảng viên hướng dẫn, nộp cuối buổi) và **một bài lab về nhà** chia ba mức Cơ bản → Trung bình → Nâng cao (nộp sau một tuần).

Buổi	Chương	Lab trên lớp	Phút	Lab về nhà
Buổi 1	1. Giới thiệu CTDL & GT	1A — Đặc tả, đếm phép toán, thực nghiệm nhân đôi n	45	1B — Tìm kiếm tuyến tính, lũy thừa nhanh, Fibonacci, 3-SUM
Buổi 1	2. Sắp xếp & tìm kiếm	2A — 5 thuật toán sắp xếp, tìm kiếm nhị phân, bảng băm	65	2B — Radix, nội suy, median-of-3, bảng băm tự cài, nghịch thế, quickselect
Buổi 2	3. Kỹ thuật thiết kế GT	3A — Sinh kế tiếp, quay lui, quy hoạch động (LCS)	55	3B — Đệ quy, quay lui, tham lam, LIS, nhánh cận TSP, chia để trị
Buổi 2	4. CTDL trừu tượng	4A — DSLK đơn, ngăn xếp, BST, BFS/DFS	60	4B — DSLK kép/vòng, hàng đợi vòng, heap, shunting-yard, Dijkstra, Kruskal, AVL

Phân bổ thời gian mỗi buổi (120 phút). Buổi 1: 10 phút giảng viên kiểm tra môi trường + 45 phút Lab 1A + 65 phút Lab 2A. Buổi 2: 55 phút Lab 3A + 60 phút Lab 4A + 5 phút nộp bài.

0.2 Chuẩn bị môi trường

- **C++:** trình biên dịch hỗ trợ **C++17** — g++ ≥ 7 (MinGW-w64 trên Windows, có sẵn trong Code::Blocks / Dev-C++ 6.x / VS Code), hoặc Visual Studio 2019 trở lên. Trong Dev-C++ cần bật `-std=c++17` ở Tools → Compiler Options.
- **Python:** Python **3.8 trở lên** (bài 3A dùng `math.comb`). Không cần cài thêm thư viện ngoài.
- Kiểm tra trước buổi học: biên dịch và chạy thử file `starter/cpp/lab1A_do_phuc_tap.cpp`. Nếu chạy được và in ra các dòng `[SAI]` là môi trường đã sẵn sàng.

```
# C++ (dong lenh)
g++ -O2 -std=c++17 -Wall -o lab1A lab1A_do_phuc_tap.cpp
./lab1A              # Windows: lab1A.exe

# Python
python lab1A_do_phuc_tap.py        # hoac: python3 ...
```

0.3 Cách làm một bài lab

1. Chọn **một** ngôn ngữ (C++ hoặc Python) cho bài trên lớp. Bài về nhà làm bằng ngôn ngữ đã chọn; làm thêm ngôn ngữ thứ hai được cộng điểm thưởng.
2. Mở file khung trong thư mục `starter/`. Mỗi chỗ cần viết code được đánh dấu `// TODO: viet code cua ban o day` (Python: `# TODO`). Đọc kỹ chú thích phía trên mỗi hàm — đó là **đặc tả** (input, output, quy tắc đếm).
3. File khung **đã biên dịch được** và có sẵn hàm `main()` kiểm thử. Khi chưa làm, chương trình in `[SAI]`; làm đúng sẽ in `[OK]`. **Không sửa `main()`** và không sửa dữ liệu test.
4. Làm lần lượt từng hàm, chạy lại sau mỗi hàm. Đừng viết hết rồi mới chạy.
5. Trả lời các **câu hỏi** cuối mỗi bài vào phiếu báo cáo (Phụ lục A). Câu hỏi chiếm một phần điểm — code chạy đúng mà không giải thích được vẫn chưa đủ.

> **Quy tắc chung.** Không dùng hàm thư viện làm sẵn cho chính thứ mà bài yêu cầu cài đặt: không gọi `sort()` trong bài sắp xếp, không dùng `heapq` / `priority_queue` trong bài tự cài heap (4B.4), không dùng `dict` / `unordered_map` trong bài tự cài bảng băm (2B.4), không dùng `list.append/pop` thay cho chỉ số `top` trong bài ngăn xếp bằng mảng (4A.2). Các chỗ được phép dùng thư viện đã ghi rõ trong đặc tả.

> **Dữ liệu ngẫu nhiên.** Mọi dữ liệu ngẫu nhiên đều sinh từ cùng một bộ sinh LCG có trong file, nên bản C++ và bản Python cho **cùng một kết quả đếm**. Chỉ các cột thời gian (ms) là khác nhau giữa các máy.

0.4 Nộp bài và đánh giá

Loại	Nộp gì	Hạn	Đặt tên
Lab trên lớp (1A, 2A, 3A, 4A)	File mã nguồn đã hoàn thành + file kết quả chạy (output.txt) + phiếu trả lời	Cuối buổi	MSSV_Lab1A.cpp / .py MSSV_Lab1A_output.txt
Lab về nhà (1B, 2B, 3B, 4B)	Như trên, nén thành một file .zip	7 ngày sau buổi học	MSSV_HoTen_Lab1B.zip

Lưu kết quả chạy ra file: ./lab1A > MSSV_Lab1A_output.txt (Python: python lab1A_do_phuc_tap.py > MSSV_Lab1A_output.txt).

Thang điểm. Mỗi lab trên lớp: 10 điểm (code chạy đúng *70%, câu hỏi* 30%). Mỗi lab về nhà: Cơ bản 5 điểm, Trung bình 3 điểm, Nâng cao 2 điểm; bài Thử thách (nếu có) cộng tối đa 1 điểm. Điểm thực hành thuộc thành phần **Bài tập 20%** của học phần.

> **Liêm chính học thuật.** Được thảo luận ý tưởng với bạn, nhưng code phải tự viết. Hai bài nộp giống nhau bất thường sẽ được giảng viên hỏi vấn đáp trực tiếp; không giải thích được code của mình thì bài nhận 0 điểm.

Phần 1. Lab trên lớp — Buổi 1 (Chương 1 và Chương 2)

Lab 1A — Đặc tả thuật toán và phân tích độ phức tạp

Mục	Nội dung
File khung	starter/cpp/lab1A_do_phuc_tap.cpp · starter/python/lab1A_do_phuc_tap.py
Thời lượng	45 phút (1.1: 10' · 1.2: 15' · 1.3: 20')
Mục tiêu	Viết đặc tả và cài đặt một thuật toán đầy đủ các trường hợp (LO1) Đếm số phép toán cơ bản và đối chiếu với công thức T(n) Ước lượng bậc độ phức tạp bằng thực nghiệm nhân đôi n

Bài 1.1. Đặc tả và cài đặt: giải phương trình $ax^2 + bx + c = 0$ (10 phút)

Mục tiêu. Từ Ví dụ 1.3 trong slide Chương 1, mở rộng thành một thuật toán **đúng và tổng quát** cho mọi bộ hệ số thực.

KIẾN THỨC CẦN NHỚ

- Một thuật toán phải có: đầu vào, đầu ra, tính xác định, tính hữu hạn, tính đúng đắn, tính tổng quát.
- Số thực dấu phẩy động không biểu diễn chính xác mọi số thập phân → không so sánh `delta == 0`, mà so sánh $|delta| \le EPS$.

ĐỀ BÀI

Cài đặt hàm `solveQuadratic(a, b, c)` (Python: `solve_quadratic`) trả về **số nghiệm** và các nghiệm: `-1` = vô số nghiệm, `0` = vô nghiệm, `1` = một nghiệm (x1 = x2), `2` = hai nghiệm với **x1 < x2**. Phải xử lý cả trường hợp **a = 0** (phương trình suy biến bx + c = 0).

HƯỚNG DẪN TỪNG BƯỚC

1. Viết đặc tả Input/Output ra giấy trước khi viết code.
2. Xử lý $a \approx 0$: nếu $b \approx 0$ thì xét c (vô số nghiệm hoặc vô nghiệm); ngược lại $x = -c/b$.
3. Tính $\Delta = b^2 - 4ac$; so sánh Δ với 0 bằng hằng `EPS` có sẵn.
4. Khi $\Delta > 0$, tính hai nghiệm rồi hoán đổi nếu cần để x1 < x2 (khi a < 0 thứ tự bị đảo).
5. Chạy chương trình: phải được `Dung 8/8 test`.

KẾT QUẢ CẦN ĐẠT

- Test `a=1 b=0.2 c=0.01` phải ra **một nghiệm kép −0.1** (không phải hai nghiệm).

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Vì sao test (1, 0.2, 0.01) cho kết quả sai nếu viết `if (delta == 0)`? Hãy in giá trị delta ra để kiểm chứng.
2. Thuật toán trong slide (chỉ xét $a \neq 0$) vi phạm tính chất nào của thuật toán khi người dùng nhập a = 0? Bản của bạn đã khắc phục ra sao?

Bài 1.2. Đếm phép toán cơ bản (phương pháp đếm) (15 phút)

Mục tiêu. Áp dụng ba bước đánh giá độ phức tạp trong slide: xác định phép toán cơ bản → đếm T(n) → xếp lớp Big-O.

ĐỀ BÀI

Cài đặt 5 hàm `countF1..countF5` chạy đúng đoạn vòng lặp mô tả trong chú thích và tăng biến đếm ở thân vòng lặp trong cùng; sau đó cài 5 hàm `formulaF1..formulaF5` trả về **công thức đóng** (không dùng vòng lặp; được dùng hàm `ceilLog2` có sẵn).

HƯỚNG DẪN TỪNG BƯỚC

1. Với mỗi đoạn, tự tính T(n) trên giấy cho n = 8 trước khi chạy (ví dụ F3 với n = 8: 7 + 6 + … + 1 = 28).
2. Cài các hàm `countF*` — chỉ là chép lại vòng lặp đã cho.

3. Cài các hàm formulaF*. F4 đếm số lần nhân đôi i bắt đầu từ 1 cho tới khi i ≥ n.
4. Chạy: cả ba dòng n = 8, 100, 1000 phải in [OK].

KẾT QUẢ CẦN ĐẠT

- Dòng n = 1000: 1000 1000000 499500 10 10000 [OK].

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Viết lớp Big-O của F1…F5.
2. F3 chỉ bằng khoảng một nửa F2. Tại sao hai đoạn vẫn cùng lớp $O(n^2)$? Liên hệ với định nghĩa Big-O (hằng số C và K).
3. Vì sao F4 với n = 1000 cho kết quả 10 mà không phải $\log_2 1000 \approx 9.97$?

Bài 1.3. Thực nghiệm nhân đôi n (doubling experiment) (20 phút)

Mục tiêu. Ước lượng bậc độ phức tạp từ thời gian chạy thật, và thấy cùng một bài toán có thể giải bằng các thuật toán $O(n^2)$, $O(n \log n)$, $O(n)$.

ĐỀ BÀI

Bài toán: cho mảng a gồm n số nguyên, **có phần tử nào trùng nhau không?** Cài ba cách: (A) hasDupBrute so sánh mọi cặp i < j, phải tăng biến toàn cục pairComparisons mỗi lần so sánh; (B) hasDupSort sắp xếp bản sao rồi xét hai phần tử kề nhau (được dùng sort); (C) hasDupHash dùng tập băm (unordered_set / set).

HƯỚNG DẪN TỪNG BƯỚC

1. Cài ba hàm, chạy để dòng Kiem tra tinh dung: [OK].
2. Quan sát bảng: cột so cap (A) phải bằng n(n−1)/2 vì dữ liệu là hoán vị (không có phần tử trùng — trường hợp xấu nhất).
3. Đọc cột tỉ số ti = T(n)/T(n/2): tỉ số ≈ 4 → $O(n^2)$; ≈ 2,1–2,3 → $O(n \log n)$; ≈ 2 → $O(n)$. Hai dòng đầu thường nhiễu do thời gian quá nhỏ.

KẾT QUẢ CẦN ĐẠT

- Bản C++ chạy n = 2000…16000, bản Python chạy n = 1000…8000 (Python chậm hơn nhiều lần).
- n = 8000 → so cap (A) = 31 996 000.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Điền bảng thời gian đo được trên máy bạn và kết luận bậc của A, B, C.
2. Dùng bảng, ước lượng thời gian của cách A khi n = 1 000 000. Nêu cách tính.
3. Vì sao chương trình dùng dữ liệu không có phần tử trùng? Nếu dữ liệu có phần tử trùng ngay đầu mảng thì thời gian A thay đổi thế nào?
4. Trường hợp xấu nhất của cách C là gì? Khi nào bảng băm mất O(1)?

Lab 2A — Sắp xếp và tìm kiếm

Mục	Nội dung
File khung	starter/cpp/lab2A_sap_xep_tim_kiem.cpp · starter/python/lab2A_sap_xep_tim_kiem.py
Thời lượng	65 phút (2.1: 25' · 2.2: 20' · 2.3: 20')
Mục tiêu	Cài đặt và đếm phép so sánh của 5 thuật toán sắp xếp trong slide (LO2) So sánh trường hợp tốt/xấu trên dữ liệu ngẫu nhiên, đã sắp, sắp ngược Tìm kiếm nhị phân có vết; bảng băm chaining và linear probing (bài tập 4 và 8 của slide)

Bài 2.1. Selection, Insertion, Bubble sort có bộ đếm (25 phút)

KIẾN THỨC CẦN NHỚ

- comps = số lần so sánh khóa tại dòng (*) trong slide; moves = số lần swap (Selection, Bubble) hoặc số lần dịch phần tử a[j+1] = a[j] (Insertion).

- Selection chỉ swap khi minPos != i; Bubble có cờ swapped để dừng sớm; Insertion chỉ đếm so sánh khi j ≥ 0.
- Khi TRACE = true, in mảng sau mỗi lượt của vòng ngoài theo mẫu pass k: ... (đã có sẵn trong khung chú thích).

ĐỀ BÀI

Hoàn thành selectionSort, insertionSort, bubbleSort. Chương trình in vết trên mảng mẫu 5 2 9 1 5 6 (bài tập 1 của slide) và bảng comps/moves với n = 1000 trên ba loại dữ liệu.

HƯỚNG DẪN TỪNG BƯỚC

1. Tự chạy tay trên 5 2 9 1 5 6 trước, ghi số so sánh và số swap của mỗi thuật toán.
2. Cài Selection; chạy và đối chiếu vết in ra với bản chạy tay.
3. Cài Insertion, rồi Bubble.
4. Đọc bảng n = 1000 và trả lời câu hỏi.

KẾT QUẢ CẦN ĐẠT

- Mảng mẫu: Selection comps = 15, moves = 4; Insertion comps = 9, moves = 6; Bubble comps = 14, moves = 6.
- Selection luôn có 499500 phép so sánh ở cả ba cột.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Vì sao Selection có cùng số phép so sánh trên dữ liệu đã sắp và sắp ngược (bài tập 2 slide)?
2. Trên dữ liệu ngẫu nhiên, moves của Insertion bằng đúng số swap của Bubble. Giải thích bằng khái niệm **nghịch thế** (cặp i < j mà a[i] > a[j]).
3. Thuật toán nào là *thích nghi* (adaptive)? Dẫn chứng bằng cột "đã sắp tăng".
4. Khi nào nên chọn Selection sort dù nó chậm?

Bài 2.2. Quick sort (Lomuto) và Merge sort (20 phút)

ĐỀ BÀI

Cài partitionLomuto, quickSort, mergeRuns, mergeSort **đúng như slide** (pivot = phần tử cuối; trong merge lấy phần tử bên trái khi bằng nhau). Đếm comps tại dòng a[j] <= pivot và a[i] <= a[j]. Chương trình so sánh số phép so sánh trên n = 1000 và đo thời gian trên 1 000 000 số (Python: 200 000) với std::sort / sorted() (bài tập 4 slide).

HƯỚNG DẪN TỪNG BƯỚC

1. Cài partitionLomuto và kiểm tra bằng tay trên 29 10 14 37 13 (ví dụ trong slide).
2. Cài quickSort đệ quy.
3. Cài mergeRuns dùng mảng tạm, rồi mergeSort.
4. Chạy, ghi lại các số đo thời gian.

KẾT QUẢ CẦN ĐẠT

- n = 1000 ngẫu nhiên: quick 12116, merge 8697 phép so sánh.
- n = 1000 đã sắp: quick 499500, merge 5044.

> **Lưu ý.** Bản Python đã đặt sys.setrecursionlimit(10000). Thử xóa dòng này và chạy lại để thấy lỗi RecursionError của quick sort trên mảng đã sắp.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Vì sao quick sort trên mảng đã sắp cần đúng n(n−1)/2 = 499 500 phép so sánh? Viết hệ thức truy hồi.
2. Merge sort trên mảng đã sắp chỉ cần 5044 phép so sánh, ít hơn trên mảng ngẫu nhiên. Vì sao?
3. Ghi thời gian ba thuật toán với N lớn. Vì sao std::sort / sorted() nhanh hơn bản tự viết?

Bài 2.3. Tìm kiếm nhị phân có vết và bảng băm (20 phút)

ĐỀ BÀI

(a) Cài binarySearchTrace in từng bước theo mẫu step k: lo=.. hi=.. mid=.. a[mid]=.. (giống ví dụ tìm 23 trong slide).

(b) Cài hai bảng băm với h(k) = k mod m: ChainingTable (mỗi ô là một danh sách, **chèn vào đầu** danh sách) và LinearProbingTable (ô trống = −1, dò (i+1) mod m). Hàm searchProbes trả về số lần thăm dò để tìm thấy hoặc kết luận không có. Chạy với bộ khóa bài tập 8 của slide 12 25 35 26 5 và bộ khóa có đụng độ 22 33 44 1 12, m = 11.

HƯỚNG DẪN TỪNG BƯỚC

1. Cài tìm kiếm nhị phân; tìm 23 phải đúng 3 bước như slide.
2. Cài chaining (push_front / insert(0, k)).
3. Cài linear probing: thử lần lượt (k % m + i) % m với i = 0, 1, 2, …
4. Khi tìm kiếm trong linear probing: dừng khi gặp đúng khóa **hoặc gặp ô trống**.

KẾT QUẢ CẦN ĐẠT

- Bộ khóa thứ hai, chaining: ô 0 là 44 -> 33 -> 22, ô 1 là 12 -> 1.
- Linear probing: [0]22 [1]33 [2]44 [3]1 [4]12 và các ô còn lại trống.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Tìm kiếm nhị phân trên mảng 10 phần tử cần tối đa bao nhiêu bước? So với công thức $\lfloor \log_2 n \rfloor + 1$.
2. Bộ khóa thứ nhất có đụng độ không? Tính hệ số tải α.
3. Với bộ khóa thứ hai, tìm khóa 23 (không có) mất bao nhiêu probe ở mỗi cách? Giải thích hiện tượng *gom cụm sơ cấp* (primary clustering).
4. Vì sao nên chọn m là số nguyên tố?

Phần 2. Lab về nhà — Chương 1 và 2

Mỗi bài về nhà có các mức **Cơ bản** (bắt buộc, 5 điểm), **Trung bình** (3 điểm), **Nâng cao** (2 điểm). Làm theo thứ tự — bài sau thường dùng kết quả của bài trước. Hạn nộp: 7 ngày sau buổi học tương ứng.

Lab 1B — Độ phức tạp thuật toán

Mục	Nội dung
File khung	starter/cpp/lab1B_ve_nha.cpp · starter/python/lab1B_ve_nha.py
Thời lượng	Về nhà · khoảng 3 giờ
Mục tiêu	Phân tích trường hợp tốt / trung bình / xấu Giải hệ thức truy hồi bằng thực nghiệm Thấy khoảng cách khổng lồ giữa các lớp độ phức tạp

Bài 1B.1. Tìm kiếm tuyến tính: ba trường hợp [Cơ bản]

ĐỀ BÀI

Cài linearSearch(a, x) trả về chỉ số đầu tiên có a[i] = x (không có → −1), tăng comps mỗi lần so sánh. Chương trình đo trường hợp tốt nhất, xấu nhất và trung bình khi tìm lần lượt mọi phần tử của một hoán vị n = 1000.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Chứng minh số so sánh trung bình khi tìm thấy là (n+1)/2.
2. Nếu xác suất x **không** có trong mảng là 50% thì số so sánh trung bình là bao nhiêu?

Bài 1B.2. Lũy thừa nhanh [Cơ bản]

ĐỀ BÀI

Cài powNaive(b, e, m) (nhân e lần) và powFast(b, e, m) **đúng khuôn** trong chú thích (bình phương liên tiếp). Đếm số phép nhân.

KẾT QUẢ CẦN ĐẠT

- 3^1000 mod (10^9+7): naive 1000 phép nhân, fast **16** phép nhân.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Viết công thức số phép nhân của powFast theo số bit và số bit 1 của e. Kiểm chứng với e = 10^6.
2. Viết hệ thức truy hồi T(e) và giải ra Big-O.

Bài 1B.3. Fibonacci ba cách [Trung bình]

ĐỀ BÀI

Cài fibRec (đệ quy thuần), fibMemoHelper (đệ quy có nhớ) — cả hai tăng calls mỗi lần gọi — và fibIter (vòng lặp, bộ nhớ O(1)).

KẾT QUẢ CẦN ĐẠT

- F(30) = 832040, fibRec cần 2 692 537 lời gọi.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Chứng minh (quy nạp) số lời gọi của fibRec(n) bằng 2F(n+1) − 1.
2. Từ cột thời gian, tính tỉ số T(32)/T(30) và so với $\varphi^2 \approx 2{,}618$ (φ là tỉ lệ vàng). Kết luận bậc của fibRec.
3. Số lời gọi của bản có nhớ theo n là bao nhiêu?

Bài 1B.4. Bài toán 3-SUM [Nâng cao]

ĐỀ BÀI

Đếm số bộ ba $i < j < k$ có $a[i] + a[j] + a[k] = 0$ (các phần tử đôi một khác nhau) bằng ba cách: threeSumCubic $O(n^3)$; threeSumBinary sắp xếp rồi tìm nhị phân phần tử thứ ba **trong đoạn k > j** — $O(n^2 \log n)$; threeSumTwoPtr sắp xếp + hai con trỏ — $O(n^2)$.

KẾT QUẢ CẦN ĐẠT

- n = 200 → 236 bộ ba; n = 1600 → 16 057 bộ ba. Ba cách phải khớp nhau.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Vì sao trong threeSumBinary chỉ được tìm trong a[j+1..n−1]? Nếu tìm trong toàn mảng thì kết quả sai thế nào?
2. Chứng minh tính đúng của phương pháp hai con trỏ (gợi ý: phát biểu bất biến vòng lặp).
3. Dùng cột b ≈ n^… để kiểm chứng ba bậc độ phức tạp. Vì sao giá trị b đo được không đúng bằng 3 và 2?

Lab 2B — Sắp xếp và tìm kiếm (nâng cao)

Mục	Nội dung
File khung	starter/cpp/lab2B_ve_nha.cpp · starter/python/lab2B_ve_nha.py
Thời lượng	Về nhà · khoảng 4 giờ
Mục tiêu	Sắp xếp không so sánh (radix), tìm kiếm nội suy Cải tiến quick sort; tự cài bảng băm có rehash Ứng dụng chia để trị của merge sort / quick sort

Bài 2B.1. Radix sort LSD [Cơ bản]

ĐỀ BÀI

Cài countingByDigit(a, exp) (sắp ổn định theo chữ số (a[i]/exp) % 10, duyệt **ngược** khi đặt phần tử) và radixSort(a, trace). Dữ liệu bài tập 6 slide: 329 457 657 839 436 720 355.

KẾT QUẢ CẦN ĐẠT

- Sau lượt exp = 1: 720 355 436 457 657 329 839.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Vì sao mỗi lượt phải ổn định? Cho phản ví dụ khi lượt hàng chục không ổn định.
2. Độ phức tạp của radix sort? Vì sao nó không mâu thuẫn với cận dưới $\Omega(n \log n)$?

Bài 2B.2. Tìm kiếm nội suy vs nhị phân [Cơ bản]

ĐỀ BÀI

Cài binarySearch và interpolationSearch (công thức pos trong slide; chống chia 0), mỗi lần đọc a[mid] / a[pos] tăng probes. So sánh số probe trung bình trên mảng phân bố đều (10^6 phần tử) và mảng lệch $a[i] = (i+1)^3$.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Giải thích vì sao nội suy chỉ cần ≈ 1 probe trên mảng đều nhưng tệ hơn nhị phân trên mảng lệch.
2. Trên mảng 10^6 phần tử, nhị phân cần tối đa bao nhiêu probe?

Bài 2B.3. Quick sort với pivot trung vị của ba [Trung bình]

ĐỀ BÀI

Cài medianOfThree (sắp ba giá trị a[lo], a[mid], a[hi], rồi đưa trung vị về cuối làm pivot) và quickSortMed3 (chỉ áp dụng khi đoạn có ≥ 3 phần tử). So sánh số so sánh với bản pivot cuối trên ba loại dữ liệu (bài tập 3 slide).

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Với mảng đã sắp, vì sao median-of-three đưa quick sort từ $O(n^2)$ về $O(n \log n)$?

2. Hãy xây dựng (hoặc mô tả) một input vẫn làm median-of-three chạy chậm.

Bài 2B.4. Bảng băm chaining tự cài: đếm tần suất từ [Trung bình]

ĐỀ BÀI

Lớp WordCounter: hàm băm đa thức h = (h·31 + mã ký tự) mod m (mod sau mỗi bước), add(w) tăng đếm, get(w), tự động rehash sang m' = 2m + 1 khi tải > 0,75. Không dùng dict / unordered_map. Chạy trên đoạn văn có sẵn, bắt đầu với m = 7.

KẾT QUẢ CẦN ĐẠT

- 75 từ, 45 từ khác nhau; từ phổ biến nhất the = 11.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Bảng đã rehash mấy lần, m cuối cùng bằng bao nhiêu, α cuối bằng bao nhiêu?
2. Vì sao rehash có chi phí O(n) nhưng chi phí trung bình mỗi lần add vẫn là O(1) (phân tích khấu hao)?

Bài 2B.5. Đếm nghịch thế bằng merge sort [Nâng cao]

ĐỀ BÀI

Cài inversionsBrute $O(n^2)$ và sortCount — merge sort có đếm: khi lấy phần tử từ nửa phải, cộng thêm số phần tử còn lại của nửa trái. So sánh trên n = 3000 rồi chạy n = 1 000 000 (Python: 200 000).

KẾT QUẢ CẦN ĐẠT

- 2 4 1 3 5 có 3 nghịch thế.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Giải thích vì sao cộng mid − i + 1 là đúng.
2. Số nghịch thế liên hệ thế nào với số swap của Bubble sort ở bài 2.1?
3. Với dãy ngẫu nhiên, kỳ vọng số nghịch thế ≈ $n^2/4$. Kiểm chứng.

Bài 2B.6. Quickselect [Nâng cao]

ĐỀ BÀI

Cài quickSelect(a, k) tìm phần tử nhỏ thứ k (k từ 1) dùng medianOfThree + lomuto, **không** sắp xếp toàn bộ mảng; chỉ đi tiếp vào một nửa chứa vị trí k−1.

CÂU HỎI (TRẢ LỜI TRONG PHIẾU BÁO CÁO)

1. Vì sao độ phức tạp trung bình là O(n) chứ không phải O(n log n)? (Gợi ý: n + n/2 + n/4 + …)
2. Trường hợp xấu nhất? Kể tên một thuật toán chọn O(n) trong trường hợp xấu nhất.

Phụ lục A. Mẫu phiếu báo cáo

Thông tin	
Họ tên — MSSV — Lớp	
Bài lab	Lab ___ (trên lớp / về nhà)
Ngôn ngữ	C++ / Python / cả hai
Số test [OK] / tổng	
Các bài đã hoàn thành	
Khó khăn gặp phải	

Câu hỏi	Trả lời
1.1-1	
1.1-2	
…	

Phần trả lời: ghi số bài – số câu (ví dụ 2.1-3), trả lời ngắn gọn, có số liệu đo được trên máy của mình khi câu hỏi yêu cầu.

Phụ lục B. Lỗi thường gặp

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
error: structured bindings only available with -std=c++17	Trình biên dịch mặc định C++11/14	Thêm -std=c++17 (Dev-C++: Tools → Compiler Options)
Chữ tiếng Việt in ra bị lỗi	Console Windows không dùng UTF-8	Chương trình chỉ in tiếng Việt không dấu; nếu tự thêm, chạy chcp 65001
RecursionError (Python)	Đệ quy sâu (quick sort trên mảng đã sắp, DFS)	Giữ dòng sys.setrecursionlimit(10000) có sẵn
Segmentation fault / chương trình dừng đột ngột	Truy cập con trỏ NULL, chỉ số ngoài mảng, pop ngăn xếp rỗng	Kiểm tra điều kiện biên; C++ dùng -fsanitize=address -g để tìm dòng lỗi
Bộ đếm lệch so với đáp án	Đếm sai chỗ (đếm cả so sánh j >= 0, đếm swap khi minPos == i…)	Đọc lại quy tắc đếm trong chú thích ngay trên hàm
Python chạy rất lâu	Vòng lặp lồng nhau trong Python chậm hơn C++ ~50 lần	Bình thường; các kích thước n trong bản Python đã được giảm sẵn